

Gölge - Dark Matter Memory

Otonom Bellek İçeriği Gizlilik ve Aktif Savunma Mimarisi / Teknik Dokümantasyon v2

1. Proje Misyonu ve Vizyonu

Gölge projesinin temel misyonu, kritik verilerin RAM üzerinde geleneksel şifreleme yöntemleriyle (veri yapısında anormal yoğunluk ve yüksek entropi anomalisi yaratan yapılar) saklanması önüne geçmektir. Sistem, veriyi doğrudan şifrelenmiş bir blok halinde tutmak yerine, onu tamamen istatistiksel bir gürültü formuna dönüştürerek bellek havuzunun geneline homojen bir şekilde yayar. Bu süreçte havuzun genel Shannon Entropisi teorik maksimum olan üst sınıra yaklaştırılır:

$$H(X) = - \sum P(x_i) \log_2 P(x_i) \approx 8.0$$

Projenin vizyonu, durağan bir koruma katmanı yerine; veriyi sürekli yer değiştiren (Moving Target Defense), olası çarpışma ve bit bozulmalarına karşı kendi kendini onarabilen (Self-Healing) ve yetkisiz erişim algılandığı anda sıfır iz politikasıyla otonom imha sürecini tetikleyen proaktif bir güvenlik organizması inşa etmektir.

2. Dosya Yapısı ve Mimari Dağılım

Uygulama; düşük seviyeli performans çekirdeği, kriptografik rastgelelik katmanları, steganografik kodlayıcılar ve gelişmiş stres testleri olmak üzere modüler bir hiyerarşide yapılandırılmıştır:

```
DarkMatterMemory/
├── src/
│   ├── core/
│   │   ├── canary_guard.py    → Aktif kanarya tuzakları ve imha protokolü
│   │   ├── lib.rs             → Donanım hızında pointer işlemleri için Rust çekirdeği
│   │   ├── memory_pool.py     → Ana bellek havuzu soyutlaması ve zero-copy arayüzü
│   │   ├── noise_generator.py → CSPRNG tabanlı yüksek entropili gürültü motoru
│   │   ├── os_memory.py       → Windows/Linux kernel seviyesi RAM tahsis modülü
│   │   └── shadow_migrator.py → MTD (Moving Target Defense) otonom göç thread'i
│   ├── crypto/
│   │   └── hardware_entropy.py → RDRAND/RDSEED donanımsal entropi kökü ve fallback
│   └── hiyerarşisi
│       ├── seed_manager.py    → PBKDF2 HMAC şifre türetme ve İnkâr Edilebilirlik katmanı
│       ├── stego/
│       │   ├── decoder.py      → Seed tabanlı rastgele koordinat çözücü motor
│       │   ├── ecc.py          → Çoğunluk Oyu (Majority Vote) hata onarım katmanı
│       │   └── encoder.py      → Rastgele dağılımlı steganografik bayt kodlayıcı
│       └── tests/
│           ├── canary_test.py  → Kanarya tuzaklarının anlık tetiklenme doğrulaması
│           ├── entropy_test.py → Shannon Entropisi ve frekans analiz doğrulama testi
│           ├── stress_test.py  → Bellek ezme ve Brute-Force direnç ölçüm paketi
│           ├── test_all_phases.py → Uçtan uca full-stack entegrasyon testi
│           └── test_stego.py   → Pearson Ki-Kare (Chi-Square) istatistiksel analizör
└── testleri
```

3. Çekirdek Özellikler ve Gelişmiş Savunma Katmanları

src/crypto/hardware_entropy.py

1. Donanım Destekli Güvenli Havuz ve Fallback Mekanizması

Sistem, OS tabanlı güvenli bellek alanlarında sanal RAM blokları tahsis eder. Havuzun ilklendirilmesi esnasında işlemcinin termal gürültüsünden beslenen donanımsal `RDRAND/RDSEED` komut setleri kullanılır. Projede kritik bir mühendislik detayı olarak **Donanımsal Entropi Fallback** yapısı kurgulanmıştır. Kodun donanım desteği olmayan mimarilerde veya kısıtlı sanal makinelerde (VM) çökmesini engellemek adına, bir `try-except` bloğu ile işletim sistemi seviyesindeki kriptografik olarak güvenli yazılımsal `os.urandom` CSPRNG kaynağına güvenli bir şekilde geri dönüş (fallback) yapması sağlanır.

src/crypto/seed_manager.py

2. Steganografik Gizleme ve İnkâr Edilebilirlik (Plausible Deniability)

Kullanıcı şifresinden PBKDF2 HMAC SHA-512 algoritmasıyla üretilen kriptografik anahtar, havuzdaki gürültü baytlarının yerini alacak deterministik ancak asimetrik dağılımlı indeks koordinatlarını hesaplar. Entegrasyonda **Plausible Deniability (İnkâr Edilebilirlik)** felsefesi benimsenmiştir. Kullanılan `salt` (tuz) parametresinin cihaz üzerinde açıkça saklanmaması, donanım kimliklerine veya dinamik protokollere bağlanması tercih edilerek, sistemin dışarıdan zorlama veya analiz altında dahi "içeride gizli bir veri bulunduğunu" matematiksel olarak reddedebilme potansiyeli korunur.

src/stego/ecc.py

3. Hata Düzeltme Kodları (Self-Healing ECC)

RAM üzerinde meydana gelebilecek bit-flip (kutup taklası) arızalarına veya harici yazılımların bellek ezip bozma girişimlerine karşı `ecc.py` modülü devrededir. Çoğunluk Oyu (Majority Vote / TMR) prensibiyle çalışan algoritma, her bir baytı belirlenen tek sayılı bir yedeklilik faktörü kadar (örneğin 3, 5 veya 7 katı) genişleterek havuza dağıtır. Bu sayede veri bloklarının yarısı manipüle edilse dahi veri kayıpsız onarılır.

src/core/shadow_migrator.py

4. Hareketli Hedef Savunması (Moving Target Defense - MTD)

Bellekte sabit duran veri, statik analizlerin hedefidir. Arka planda daemon modunda koşan `ShadowMigrator` thread'i, belirlenen periyotlarda (örn. her 5-10 saniyede bir) gizlenen veriyi ECC katmanıyla çözüp çeker, bellek havuzunu donanımsal rastgelelikle tamamen sıfırlar ve ardından veriyi yeni hesaplanan ofset koordinatlarına yeniden gömer. Veri RAM içinde sürekli hareket ederek sabit hedef olmaktan çıkar.

src/core/canary_guard.py

5. Aktif Kanarya Tuzakları ve Otonom İmha Protokolü

Bellek bloğunun stratejik lokasyonlarına hassas kontrol baytlarından oluşan kanarya tuzakları yerleştirilir. `CanaryGuard`, pool üzerindeki her yazma işleminden sonra tuzak noktalarını izler. Belleğe yönelik yetkisiz bir "memory dump" veya doğrudan modifikasyon girişimi tespit edildiği anda milisaniyeler içinde `_self_destruct()` tetiklenir. Tüm havuz fiziksel rastgelelikle ezilerek sıfırlanır ve `os._exit(1)` ile süreç sonlandırılarak geride tek bir bayt iz bırakılmaz.

src/core/memory_pool.py & lib.rs

6. Çekirdek Performans ve Zero-Copy Optimizasyonu

Gelişmiş bellek işlemlerinin performanstan ödün vermeden (near-zero latency) çalışması için iki aşamalı mimari optimizasyon uygulanmıştır. İlki, `lib.rs` üzerinden Rust entegrasyonu ile C/C++ hızında Raw Pointer (doğrudan bellek adresi) yönetimidir. İkincisi ise Python tarafında bellek kopyalama maliyetlerini (overhead) tamamen ortadan kaldıran **Zero-Copy** tasarımıdır. `MemoryPool.get_view()` fonksiyonu, veriyi RAM üzerinde kopyalamadan doğrudan referans üzerinden sıfır gecikmeyle okuma ve yazma işlemlerine açan bir `memoryview` objesi döndürür. Bu, büyük veri bloklarında CPU yükünü minimize eden kritik bir performans bileşenidir.

4. Güvenlik Testleri ve Kapasite Doğrulamaları

Proje kapsamındaki kararlılık ve savunma mekanizmaları, `tests/` dizinindeki yıkıcı stres testleriyle doğrulanmıştır:

- **İstatistiksel Görünmezlik ve Ki-Kare:** 100 KB boyutunda ham veri gömüldükten sonra havuz entropisindeki değişimin `0.0000011` seviyesinde kaldığı ve Pearson Ki-Kare (Chi-Square) dağılım testlerini tam başarıyla geçtiği, verinin dışarıdan anlamsız bir gürültüden ayırt edilemeyeceği tescillenmiştir.
- **Agresif Çarpışma ve Bellek Ezme:** Havuz bütünlüğünün %5 ila %10'unun kasıtlı olarak rastgele harici verilerle ezildiği senaryolarda, ECC katmanı sayesinde orijinal verinin tek bir karakter hatası olmaksızın geri döndürülebildiği kanıtlanmıştır.
- **Seed Brute-Force Direnci:** Kaba kuvvet saldırı senaryolarını test eden `stress_test.py` kapsamında, sisteme ardışık 1.000 adet hatalı şifre/anahtar enjekte edilmiştir. Sistem bu süreçte istikrarsızlığa uğramak veya çökmek yerine, **Fail-Safe** koruma mekanizmasını devreye alarak hatalı istekleri kontrollü hata fırlatma yapılarıyla (Geçersiz veri uzunluğu tespiti vb.) güvenli bir şekilde reddetmiş ve brute-force girişimlerini tamamen bloke etmiştir.
- **Anlık Kanarya İmha Tepkisi:** Saldırganın doğrudan kanarya kontrol adreslerine yazma yaptığı an, koruma mekanizmasının gerçek zamanlı (real-time) müdahale hızı ölçülmüş, milisaniyeler mertebesinde imha protokolünün başarıyla koşturulduğu doğrulanmıştır.

--- Teknik Dokümantasyon Sonu ---